Program: BS Computer Science
Semester: Fall 2025
Credit Hour(s): 1
Pre-requisite(s): EE1005 Digital Logic Design
Instructor: Engr. Muhammad Hassaan Bin Shoukat – hassaan.shoukat@nu.edu.pk

Lab # 6: Conditional Jumps, Unconditional Jumps and Relative Addressing

1. Objectives

- Understand and use different **conditional jump instructions**.
- Implement **unconditional jumps**.
- Apply **relative addressing mode** in jump instructions.

2. Jumps

Jump instructions transfer program control to a different location in the code.

2.1. Flags Involved in Jumps

- ZF (Zero Flag)
- SF (Sign Flag)
- CF (Carry Flag)
- OF (Overflow Flag)

2.2. Unconditional Jump

The *JMP* instruction transfers control to a specified label unconditionally, without checking any flags.

Syntax: `JMP LABEL`

2.3. Conditional Jumps

Conditional jumps check the flag register and only jump if the condition is met. They are grouped based on the type of comparison:

A. Based on Single Flags:

Instruction	Condition	Description	Flags Checked
JE	Jump if **Equal**	Jumps if the operands of the preceding comparison were equal	ZF = 1
JNE	Jump if **Not Equal**	Jumps if the operands were not equal	ZF = 0

JNZ	Jump if **Not Zero**	Jumps if the result of the preceding operation was not zero	ZF = 0
JC	Jump if **Carry**	Jumps if **CF** is set.	CF = 1
JO	Jump if **Overflow**	Jumps if **OF** is set.	OF = 1
JS	Jump if **Sign**	Jumps if **SF** is set (result is negative).	SF = 1
JNS	Jump if **No Sign**	Jumps if **SF** is clear (result is non-negative).	SF = 0

B. Based on Signed Comparisons:

Instruction	Condition	Description	Equivalent to
JG	Jump if **Greater**	Jumps if destination **>** source	**JLE** (Jump if Not Less than or Equal)
JL	Jump if **Less**	Jumps if destination **<** source	**JNGE** (Jump if Not Greater than or Equal)
JGE	Jump if **Greater or Equal**	Jumps if destination **≥** source	**JNL** (Jump if Not Less than)
JLE	Jump if **Less or Equal**	Jumps if destination **≤** source	**JNG** (Jump if Not Greater than)

C. Based on Unsigned Comparisons:

Instruction	Condition	Description	Equivalent to
JA	Jump if **Above**	Jumps if destination > source (Unsigned)	**JNBE** (Jump if Not Below or Equal)
JB	Jump if **Below**	Jumps if destination < source (Unsigned)	**JNAE** (Jump if Not Above or Equal)
JAE	Jump if **Above or Equal**	Jumps if destination ≥ source (Unsigned)	**JNB** (Jump if Not Below)
JBE	Jump if **Below or Equal**	Jumps if destination ≤ source (Unsigned)	**JNA** (Jump if Not Above)

2.4. Relative Addressing in Jumps

In modern assembly, most jump instructions use relative addressing.

- Instead of jumping to a fixed or absolute memory address, the program jumps to an address **relative** to the **current instruction pointer**.
- Target address is calculated as:

  Target = Current Instruction Pointer (IP) + Displacement (Offset)
- Displacement is usually 8-bit or 16-bit signed value, so jumps can go forward or backward.

Example:

```
     JMP +10         ; Jump 10 bytes forward
     JMP -5          ; Jump 5 bytes backward
     JMP NEXT        ; CPU adds displacement to IP to reach NEXT
```

3. Examples

Example 1:

```
[org 0x0100]
     mov cx, 5
     mov ax, 0

     start_loop:
          add ax, 2
     loop start_loop   ; Uses relative addressing
mov ax, 0x4c00
int 0x21
```

Example 2:

```
[org 0x0100]
    mov ax, 5
    mov bx, 3
    mov cx, 0
    mov dx, 0

    cmp ax, bx

    je  case_equal         ; Jump if Equal (ZF=1)
    jne case_not_equal     ; Jump if Not Equal (ZF=0)
    ; AX=5, BX=3 → ZF=0 → goes to JNE

    case_equal:
        mov cx, 1111h          ; Marker for Equal
        jmp continue

    case_not_equal:
        mov cx, 2222h          ; Marker for Not Equal

    continue:             ; Greater/Less comparisons (signed)
        cmp ax, bx
        jg  case_greater       ; Jump if Greater (signed)
        jle case_less_equal    ; Jump if Less or Equal (signed)

    case_greater:
```

```
    mov dx, 3333h
    jmp unsigned_check

case_less_equal:
    mov dx, 4444h

; Unsigned comparisons

unsigned_check:
    cmp ax, bx
    ja  case_above        ; Jump if Above (unsigned >)
      ; CF = 0 AND ZF = 0
    jb  case_below        ; Jump if Below (unsigned <)
      ; CF = 1

case_above:
    mov byte [result1], 55h
    jmp sign_check

case_below:
    mov byte [result1], 66h

; Sign flag test

sign_check:
    mov ax, -2
    test ax, ax
    js  case_negative     ; Jump if Sign = 1
    jns case_non_negative ; Jump if Sign = 0

case_negative:
    mov byte [result2], 77h
    jmp overflow_check

case_non_negative:
    mov byte [result2], 88h

; Overflow flag test

overflow_check:
    mov al, 127
    add al, 1
    jo  case_overflow
    jno case_no_overflow
```

```
    case_overflow:
        mov byte [result3], 99h
        jmp end_prog

    case_no_overflow:
        mov byte [result3], 0AAh

    ; End Program
    end_prog:
        mov ax, 0x4c00
        int 0x21

result1: db 0
result2: db 0
result3: db 0
```

4. Lab Tasks

Task 1

Write a program to compare two numbers stored in registers. If they are equal, store 1 at memory location **num**, otherwise store 0.

Task 2

Write a program that checks if a number is positive, negative, or zero using *conditional jumps*. Store results in memory.

Task 3

Write a program to implement a simple countdown from 5 to 1 using JNZ.